SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing **COURSE CODE: CSA3711**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3

4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3
5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3

9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3

15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Code of Conduct:

*I **RenugaDevi .B(192311111)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: RenugaDevi .B

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

1. Testing as an Engineering Activity – Banking Application

Software testing should follow a planned and measurable method instead of relying on random checks. For a banking application, the team should list important functions such as login, balance enquiry, fund transfer and bill payment, then prepare test cases from the requirements.

A traceable process can be followed: Requirement → Risk → Test Case → Execution Result → Defect. Equivalence partitioning, boundary value analysis, decision tables and state-transition testing help select useful test inputs. Automation tools can repeat regression tests, while a common documentation standard keeps test records consistent.

For example, fund transfer testing should cover zero and negative amounts, transfer limits, insufficient balance, an invalid beneficiary and a network interruption. Risk-based testing, model-based testing, boundary testing and automated regression testing are more reliable than ad-hoc testing because they focus on important conditions and make results repeatable.

2. Structured Testing Process for ATM Withdrawal

ATM withdrawal must be checked at different stages of the Software Development Life Cycle (SDLC), not only after coding is complete.

• Requirements: define PIN rules, available-balance conditions and daily withdrawal limits.

• Design: prepare scenarios for card reading, authentication and cash dispensing.

• Unit testing: check PIN validation and withdrawal calculations separately.

• Integration testing: verify communication among the ATM, bank server and cash dispenser.

• System testing: test the complete withdrawal journey.

• Regression testing: repeat important cases after a change.

• Acceptance testing: confirm that the ATM meets user and banking requirements.

For instance, a withdrawal of Rs. 5,000 should succeed when the balance and limits permit it, while an amount above the daily limit should be rejected. A consistent process prevents missed stages, makes results repeatable and helps reduce defects reaching customers.

3. Error, Defect and Failure in Login and Transactions

An error is a human mistake, a defect is a flaw introduced into the software, and a failure is the incorrect behaviour observed when the software runs.

For example, a programmer may use an assignment operator instead of a comparison operator while checking a password. The coding mistake is the error; the incorrect statement in the program is the defect; and allowing an unauthorised login is the failure.

The relationship is: Human error → Software defect → Incorrect execution → Observed failure. In a transaction, an incorrect balance-calculation formula may be the defect, while displaying the wrong balance is the failure.

Reports such as “login is not working” are too vague. A useful defect report should include the expected result, actual result, steps to reproduce, test environment, severity and supporting evidence. Clear terminology helps the team investigate and correct the real cause.

4. Defect Clustering and Test Prioritization

Defect clustering means that a large share of defects is often found in a small number of modules. Therefore, testing effort should be based on risk and defect history rather than divided equally.

Suppose an online shopping application has repeated defects in payment, checkout and discount calculation. Those areas should receive more testing than a low-risk page such as About Us. A practical priority estimate can consider business risk, previous defect count and how frequently a feature is used.

A suitable order could be payment processing, checkout, login, product search and then informational pages. If previous releases failed during festival discounts, those discount scenarios should be added to the regression suite before the next sale.

Other principles, such as the impossibility of exhaustive testing and the need to refresh test cases, also guide planning. Prioritization directs limited time toward areas where failures would be most likely or most harmful.

5. Effective Use of a Defect Repository

A defect repository should be used as a learning resource for future releases, not simply as a place to store old bug reports. Defects can be grouped as functional, interface, performance, security, data, compatibility or logic defects.

For example, if several releases contain incorrect tax calculations, repository analysis may show that the same pricing component is responsible. Testers can then create regression cases for tax calculations, boundary values and related pricing rules.

Each record should include the affected module, defect type, severity, root cause, release, frequency, status and resolution. Reviewing this information helps the team identify repeated patterns and improve future test planning.

To prevent recurrence, the team should perform root-cause analysis and turn important past defects into permanent regression tests. This approach preserves knowledge, improves coverage and reduces the chance of the same issue returning.

6. Mobile Banking Fund Transfer Crashes

A crash during a mobile banking transfer should be investigated under different conditions instead of repeating only a normal successful transfer. The tester should vary the transfer amount, account balance, beneficiary status, network strength, transaction frequency, device model, operating-system version and whether the app is in the foreground or background.

For example, the application might crash if the connection changes from 4G to 5G while the transaction is being confirmed. Crash logs, transaction IDs and device details can help identify the condition that caused the problem.

The investigation should follow this sequence: Reproduce the issue → Isolate the condition → Collect evidence → Record the defect → Fix → Retest → Run regression tests.

Functional, negative, stress, interruption and compatibility testing should be included. Testing these situations helps uncover problems beyond the normal user journey and improves the reliability of banking transactions.

7. Testing and Debugging

Testing and debugging are connected activities, but they have different purposes. Testing checks whether the software behaves as expected and reports any mismatch. Debugging investigates the cause of that mismatch and changes the code to correct it.

For example, if an ATM dispenses Rs. 2,000 when the customer requests Rs. 1,000, the tester repeats the scenario, records the actual and expected results, and raises a defect. The developer then examines the withdrawal logic, finds the incorrect calculation and applies a fix.

A problem occurs when a developer assumes that fixing one defect proves the whole system is correct, or when the defect is changed without proper tracking. The recommended cycle is: Test → Report → Debug → Fix → Retest → Regression test → Close.

Keeping these responsibilities clear improves accountability, while communication between testers and developers ensures that the fix is verified and does not introduce new problems.

8. Cross-Browser Testing for E-Commerce

An e-commerce website may behave differently across browsers because rendering engines, JavaScript support, CSS handling and security settings are not identical. A planned browser-compatibility matrix should be prepared instead of checking pages randomly.

The matrix should cover important functions such as login, product filtering, cart updates and payment in browsers such as Chrome, Edge, Firefox and Safari. The main customer journey—login, search, select a product, add it to the cart and pay—should receive priority.

Selenium or Playwright can automate repeated browser checks. Cloud browser services can help test different browser and operating-system combinations. When a problem is found, the report should state the browser and version, operating system, screen size, reproduction steps and screenshot or video.

This approach makes compatibility defects easier to reproduce, increases coverage and helps provide a consistent shopping experience across supported browsers.

9. Classification of Payroll Logic Defects

Payroll defects should be classified by their cause and effect so that the right test technique can be selected. For example, net salary may be calculated as:

Net Salary = Basic Salary + Allowances – Deductions

If the program subtracts allowances instead of adding them, it contains a logic defect.

Common categories include calculation defects (incorrect arithmetic), boundary defects (incorrect results near salary or tax limits), data defects (wrong employee details), requirement defects (incorrect interpretation of tax rules) and integration defects (incorrect attendance data transfer).

Classification helps determine the testing approach. Calculation defects need tests with independently worked-out expected values, while boundary defects need boundary value analysis. Automated payroll regression tests should be run after changes, especially to salary and tax rules. These practices reduce the risk of incorrect salary payments being introduced in later releases.

10. Replacing Random Testing with Designed Test Cases

Random testing may reveal some issues, but it does not show whether important requirements have been covered. A better method is to turn each requirement into test conditions and then prepare documented test cases.

For a password field, cases should include an empty value, the minimum and maximum allowed lengths, values just below and above the limits, special characters, an incorrect password and a correct password.

Equivalence partitioning, boundary value analysis, decision tables and state-transition testing can provide systematic coverage. Each test case should have a unique ID, preconditions, input data, expected result and a link to the relevant requirement.

The goal is not simply to increase the number of tests. It is to select meaningful tests that cover important conditions and are likely to reveal defects. This makes execution more organized and test results easier to evaluate.

11. Data Inconsistency in a Healthcare Application

Healthcare applications share patient information across registration, appointments, billing and medical records. If one module is updated but another continues to display an old value, the cause may be an integration, database synchronization or transaction-management defect.

Testing should trace the information through the complete path: Input → Database → Service/API → Receiving module → Display. The same test patient record can be checked at each stage to confirm that the values match. Testers should also check simultaneous updates, interrupted transactions and rollback behaviour.

The defect repository should record the affected module, field, root cause and related interface. Reports should use a test patient ID and include the old value, new value, expected behaviour and evidence, without exposing real patient information.

Systematic data-integrity testing helps detect inconsistencies early and reduces the risk of incorrect information spreading between modules.

12. Preventing Repeated Defects Across Releases

When the same defects return in multiple releases, the team may not be using lessons from earlier testing. A defect repository can reveal recurring modules, root causes and categories that need stronger coverage.

For example, repeated login-timeout defects may require more regression tests; payment failures may call for stronger integration testing; discount-calculation problems may need boundary tests; and mobile interface defects may require a wider range of devices.

The team should review defect frequency, severity, affected release, root cause and resolution. High-frequency or high-impact issues should become permanent regression scenarios. The process is: Previous defect → Repository → Pattern analysis → Additional test → Verification → Reduced recurrence.

Using repository data in test planning makes the repository a quality-improvement tool rather than only a reporting record. It supports better prevention and more consistent results in future releases.

13. Learning System Failure During Peak Usage

A learning platform may work well with a small class but fail when thousands of students use it together. This indicates that normal functional tests did not represent real usage levels.

The test plan should include concurrent logins, simultaneous video access, quiz submissions, database load, network delays and session expiry. Load testing can increase the number of users gradually and measure response time, error rate and system resources.

For example, testing with only 50 users is insufficient if 5,000 students must submit examination answers within five minutes. Load testing checks expected demand, stress testing identifies the system's breaking point, spike testing checks sudden increases, and endurance testing checks sustained usage.

Including these tests before deployment helps identify capacity limits and performance bottlenecks. This improves availability and reliability during actual examinations.

14. Early Tester Involvement in the SDLC

Testers should participate from the beginning of development instead of joining only after coding is complete. During requirements analysis, they can identify unclear rules; during design, they can review interfaces and testability; and during development, they can prepare test data and automation.

For example, a requirement may state that an account is locked after “multiple” incorrect passwords. A tester should ask whether the limit is three, five or ten attempts and whether a successful login resets the counter. Clarifying this before implementation avoids misunderstandings and later rework.

This practice is known as shift-left testing. It encourages analysts, developers, testers and business stakeholders to discuss quality throughout the SDLC. Early involvement can reduce correction costs, improve requirement clarity and help prevent defects before they become part of the finished application.

15. Incomplete Test Coverage

A system can pass many tests and still fail if those tests cover only normal conditions. For an online payment system, testing a valid card, correct OTP and sufficient balance is not enough.

Additional cases should include an expired card, incorrect OTP, loss of network, duplicate payment, session timeout and payment-gateway failure. A coverage matrix can map each requirement to its test cases and reveal missing areas. Risk-based prioritization helps the team address gaps that could cause the greatest harm.

Equivalence partitioning, boundary value analysis, decision tables, negative testing and exploratory testing can strengthen the test suite. Testers should cover important inputs, states, error conditions and complete user journeys.

Effective coverage is about testing the right conditions, not merely counting test cases. Reviewing coverage before release helps reduce the risk of hidden defects reaching users.

16. Improving Defect Reporting in a Logistics Application

A defect report should contain enough detail for another team member to reproduce the problem without asking the tester to explain it again. A statement such as “tracking is not working” does not provide sufficient information.

A structured report should include a defect ID, title, environment, preconditions, reproduction steps, expected result, actual result, severity, priority and evidence. For example, it may state that entering a valid tracking number displays “Shipment not found” even though the shipment exists in the backend.

Terminology should also be used consistently: a defect is a flaw in the software, while a failure is the incorrect behaviour seen during execution. Clear reports improve communication between testers, developers and project managers. Easier reproduction helps the team diagnose, fix and verify issues more quickly.

17. Process-Based Testing Across Modules

Testing becomes inconsistent when each module is checked using a different personal method. The team should establish a common workflow while allowing specialized tests for individual modules.

A standard workflow is: Requirement analysis → Test planning → Test design → Test-data preparation → Execution → Defect reporting → Retesting → Regression testing → Closure.

For example, the login, cart and payment teams in an e-commerce project should use the same test-case format, defect severity scale and execution statuses. Shared templates, checklists, entry and exit criteria, and peer reviews help maintain consistency.

A defined process makes results easier to compare and repeat. It also helps a new tester understand the project without depending entirely on another person's individual testing style. Standardization improves coordination and gives the team clearer evidence of testing progress.

18. Deployment Failure Due to Edge Cases

Edge cases are often missed when testing focuses only on common input values. Boundary value analysis is useful because defects frequently occur at the limits of an allowed range.

For a ticket-booking system that accepts seat numbers from 1 to 100, tests should include 0, 1, 2, 99, 100 and 101. For a transaction limit of Rs. 1 to Rs. 50,000, useful values are I0, Rs. 1, I2, I49,999, Rs. 50,000 and I50,001.

These values check the lower boundary, upper boundary and values immediately around them. Testers should identify limits from requirements and business rules before execution. Invalid values should be rejected with clear messages, while valid boundary values should be handled correctly.

Important edge cases should be saved as regression tests so that future code changes do not reintroduce the same problems. This reduces the chance of boundary-related failures after deployment.

19. Defect Repository for Poor Documentation

A defect repository provides a shared record of software problems and their resolution. Each entry should explain what happened, where and when it happened, who reported it, how serious it was, why it occurred, how it was fixed and whether the fix was retested.

Instead of recording only “Payment bug – fixed,” the report should identify the affected release and payment method, list reproduction steps, describe the root cause and show the verification status. Useful fields include defect ID, module, type, severity, priority, root cause, status, release and resolution.

Structured records help the team discover patterns, such as repeated critical defects in payment integration. The information can guide future test planning, support root-cause analysis and help prevent recurrence.

Consistent documentation improves traceability and communication, and ensures that knowledge about earlier defects is not lost when team members or releases change.

20. Redesigning Testing for Banking Transaction Failures

Repeated banking transaction failures should be addressed by testing the complete transaction lifecycle instead of checking screens separately. The flow includes login, beneficiary selection, amount entry, validation, authorization, debit, server confirmation, receipt generation and balance update.

Each step should be tested with normal inputs, invalid inputs and interruptions. For example, if the network disconnects immediately after the account is debited, the system must not leave the user charged while showing the transaction as failed. It should recover safely and provide a consistent transaction status.

The test suite should include functional, negative, boundary, API/integration, security, recovery, concurrency and automated regression testing. Risk-based planning should give money movement higher priority than low-risk features such as profile customization.

Testing the complete lifecycle and checking transaction states helps expose financial inconsistencies early. This structured approach improves reliability and reduces the likelihood of repeated transaction failures reaching customers.